CASCADE: Autonomous Orbital Collision Avoidance for Congested LEO

National Space Hackathon 2026 — IIT Delhi

Team Space_Invaders

https://github.com/Aqwerty321/PROJECT-B.O.N.K

March 2026

Abstract

Low Earth Orbit faces an escalating Kessler-syndrome risk as mega-constellation deployments collide with a legacy debris population exceeding 27,000 tracked objects. We present **CASCADE**, a deterministic, single-container Autonomous Constellation Manager that ingests high-frequency ECI telemetry, screens for conjunctions through a multi-tier filtering cascade, and autonomously schedules collision-avoidance and recovery burns under fuel, cooldown, and line-of-sight constraints. The engine processes 50 satellites against 10,000 debris fragments in **13.3 ms** mean tick latency with **zero false negatives** across all test families. A React/Canvas/WebGL frontend delivers a mission-console experience at 60 FPS. CASCADE ships as a single Docker image (`ubuntu:22.04`, port 8000) with 8/8 CTest safety gates enforced in CI, meeting every deployment and API contract specified in the problem statement.

1. Introduction and Design Goals

The National Space Hackathon 2026 challenges teams to build an Autonomous Constellation Manager (ACM) that can (i) ingest high-frequency ECI state vectors, (ii) predict conjunctions up to 24 hours ahead without $O(N^2)$ bottlenecks, (iii) autonomously compute and schedule evasion and recovery maneuvers, and (iv) visualize the operational state in a real-time mission dashboard.

CASCADE addresses these objectives under a strict *safety-first* design philosophy:

- **Zero false negatives**: every ambiguous pair is escalated to a more expensive solver, never silently dropped.
- **Deterministic runtime**: identical inputs always produce the same maneuver sequence, enabling reproducible CI gates.
- **Conservative classification**: the engine prefers false positives (unnecessary screening work) over false negatives (missed collisions).
- **Single-container deployment**: one `docker build` yields a self-contained binary and frontend, exposed on port 8000.

The remainder of this report details the system architecture (§2), key design decisions and trade-offs (§3), numerical methods (§4), frontend console (§5), validation evidence (§6), and deployment pipeline (§7).

2. System Architecture

Figure 1 illustrates the end-to-end data path. The system is a single-process C++20 engine linked against two FetchContent dependencies — `cpp-httplib` v0.15.3 (HTTP) and `simdjson` v3.9.4 (JSON parsing) — with Boost headers for version verification.

```
POST /api/telemetry
  ↓
[State Store] - lock-free SoA upsert
  ↓
[Propagation] - adaptive RK4+J2
  ↓
[Broad Phase] - O(N log N) SMA+incl bins
  ↓
[Narrow Phase] - MOID + TCA + RK4 refine
  ↓
[CDM Engine] - 24 h conjunction horizon
  ↓
[COLA Planner] - RTN evasion + Tsiolkovsky
  ↓
[Recovery] - CW/ZEM slot-targeting burn
  ↓
REST API (port 8000)
```

Figure 1: End-to-end simulation tick pipeline.

2.1 State Representation

All objects are stored in a Structure-of-Arrays (SoA) layout indexed by string identifier. Each record holds:

- 6-DOF ECI state vector $(\vec{r}, \vec{v})$ in km and km/s
- Precomputed Keplerian elements $(a, e, i, \Omega, \omega, M)$
- Radial bounds $(r_{\min}, r_{\max})$ for shell overlap
- Fuel mass, cooldown timer, slot reference, burn history

2.2 API Surface

The engine exposes five primary endpoints on port 8000, bound to `0.0.0.0`:

Endpoint	Purpose
`POST /api/telemetry`	Bulk ECI state ingestion
`POST /api/maneuver/schedule`	Evasion/recovery burn submission
`POST /api/simulate/step`	Advance simulation by Δt s
`GET /api/visualization/snapshot`	Geodetic snapshot for frontend
`GET /api/status`	Engine health & diagnostics

Table 1: Primary REST API endpoints.

Additional debug endpoints (`/api/debug/{burns, conjunctions, propagation, conflicts}`) expose internal ring buffers for observability.

3. Design Decisions and Trade-offs

Several non-obvious architectural choices emerged from iterative development over 60+ commits. We document the key trade-offs to explain why the system takes its current form.

3.1 Julia Bridge → Pure C++ Runtime

The architecture document originally specified a Julia/`jluna` hybrid for heavy numerics (`DifferentialEquations.jl`, analytical MOID). We implemented a complete NSGA-II parameter tuner via `jluna` (`tuner/nsga2_tuner.jl`) and verified end-to-end optimization over 69 parameters. However, the Julia 1.10 runtime added **∼200 MB** to the Docker image and introduced JIT latency on container boot. Since the grading VM runs a cold `docker build`, we removed the Julia runtime from the production Dockerfile and re-implemented all numerical kernels in pure C++20. The NSGA-II tuner remains available as an offline tool (`PROJECTBONK_ENABLE_JULIA_RUNTIME` CMake guard) but the submission binary has zero Julia dependency.

3.2 Shadow-First Filter Rollout

New narrow-phase filters (plane/phase gate, MOID proxy) are deployed in *shadow mode* by default: the gate evaluates and logs rejection statistics but does not actually discard pairs. This allows us to calibrate false-negative risk against the full RK4 reference before activating hard filtering. The pattern was motivated by an early observation that the D-criterion gate's threshold was sensitive to orbital element quality — activating it prematurely on poorly-determined debris could violate the zero-false-negative invariant. Shadow-first rollout ensures every filter accumulates calibration evidence (exposed via `/api/status?details=1`) before moving to production.

3.3 Heuristic → CW/ZEM Recovery Solver

The initial recovery planner used proportional RTN gains (`HEURISTIC` mode) — simple but producing sub-optimal ΔV vectors, especially on eccentric orbits where velocity-direction diverges from true tangential. An audit identified three correctness issues: (i) mean motion computed from instantaneous radius instead of semi-major axis, (ii) $\hat{t}$ approximated as $\hat{v}$ instead of $\hat{n} \times \hat{r}$, and (iii) simplified linear CW terms instead of true $\sin(n\tau)/\cos(n\tau)$ state-transition matrix. All three were fixed, and `CW_ZEM_EQUIVALENT` became the default solver, with the heuristic retained as a near-singularity fallback.

3.4 3D Globe → 2D Operations Dashboard

The frontend initially featured a full-viewport Three.js globe with Fresnel atmosphere, glassmorphic HUD panels, and a "TRON × Star Wars" cockpit aesthetic. While visually impressive, judge feedback priority analysis revealed that the PS requires a *2D operational dashboard* with ground tracks, maneuver timelines, and resource heatmaps. We pivoted: the 3D globe was demoted to a supporting context element, and a Canvas-based Mercator ground track with 90-minute trail/forecast, live terminator, and a scrollable Gantt maneuver timeline became the primary surfaces. This decision improved judge-rubric alignment (Section 6 compliance) at the cost of visual spectacle.

3.5 MOID: Sampled Proxy vs. Analytical Solver

The problem statement rewards algorithmic sophistication, so we initially targeted a Gronchi (2005) analytical quartic MOID solver. We implemented it (`orbit_math.cpp`) and observed correct results, but the analytical path's sensitivity to near-singular orbital frames (e.g., circular orbits where argument of perigee is undefined) introduced edge-case fragility. We adopted a pragmatic compromise: a 72×72 true-anomaly grid proxy (5,184 evaluations) with a 2.0 km reject threshold, deployed in shadow mode. The sampled proxy is robust to all orbit types and provides a reliable pre-filter ahead of the expensive RK4 refinement stages.

3.6 All-Pairs Fail-Open: Why Not Tiered Fallback?

When propagation fails for an object (producing NaN or divergent states), the engine must decide how to screen that object. We investigated a tiered fallback (e.g., use last-known elements for a reduced broad-phase check) but proved that the broad phase cannot safely pair crossing-orbit objects when elements are stale. The only guaranteed zero-false-negative strategy is to check the failed object against *all* debris via the full narrow-phase pipeline. This costs more compute but eliminates

the risk of silently missing a conjunction — consistent with the safety-first design philosophy.

4. Numerical Methods

4.1 Orbital Propagation

The propagator implements an adaptive three-mode scheme:

Mode 1 — J2 Secular Fast Path. For low-eccentricity ($e \leq 0.02$), high-perigee ($\geq 500\,\text{km}$) objects with $\Delta t \leq 30\,\text{s}$, we apply closed-form J2 secular rates to mean elements:

$$\dot{\Omega} = -\frac{3}{2} n J_2 \left(\frac{R_E}{p}\right)^2 \cos i \tag{1}$$

$$\dot{\omega} = \frac{3}{4} n J_2 \left(\frac{R_E}{p}\right)^2 (5\cos^2 i - 1) \tag{2}$$

where $n = \sqrt{\mu/a^3}$, $p = a(1-e^2)$, and $R_E = 6378.137\,\text{km}$. An extended fast lane admits $e \leq 0.003$, perigee $\geq 650\,\text{km}$, $\Delta t \leq 45\,\text{s}$. Long horizons are sub-stepped at 300 s intervals.

Mode 2 — Probe-Then-Escalate. Objects outside the fast lane first receive a J2-secular prediction, then a coarse RK4 probe (max step 120 s) validates the result. If drift exceeds 0.5 km in position or 0.5 m/s in velocity, the engine escalates to full RK4.

Mode 3 — Full RK4+J2. Forced for $\Delta t > 21{,}600\,\text{s}$ (6 h), perigee $< 350\,\text{km}$, or $e \geq 0.98$. Classical fourth-order Runge–Kutta with maximum substep 60 s, integrating the acceleration:

$$\vec{a}_{J2} = \frac{3}{2} \frac{J_2\, \mu\, R_E^2}{r^5} \begin{bmatrix} x(5z^2/r^2 - 1) \\ y(5z^2/r^2 - 1) \\ z(5z^2/r^2 - 3) \end{bmatrix} \tag{3}$$

Finite-value guards after every substep prevent NaN propagation.

Validation. Maximum position error at 86,400 s (a full day) is $<$ **0.061 km**, verified by regression gate `phase2_regression_gate`.

Why not RKF45 or Dormand–Prince? Higher-order adaptive integrators would improve accuracy per step, but the collision threshold (100 m) is orders of magnitude larger than the RK4 truncation error at 60 s substeps. RK4 avoids the overhead of error estimation and step rejection logic, keeping the per-object cost predictable — critical for deterministic tick budgets. The probe-then-escalate scheme already provides adaptivity at the architectural level.

4.2 Broad-Phase Candidate Generation

The broad phase reduces the $O(N^2)$ all-pairs space through three sequential filters:

1. **SMA–Inclination Band Indexing.** Objects are binned by semi-major axis ($\Delta a = 500\,\text{km}$) and inclination ($\Delta i = 20°$). Flat sorted arrays with binary search replace hash maps for cache locality. Neighbor search extends ± 2 bins in each dimension.
2. **Shell Overlap Test.** Perigee $r_{\min} = a(1-e)$ and apogee $r_{\max} = a(1+e)$ are compared with a 50 km margin. Objects with invalid elements fall back to instantaneous radius $\pm 200\,\text{km}$ — an intentional over-approximation that preserves the zero-false-negative guarantee.
3. **D-Criterion Gate.** A simplified Drummond metric:

$$D = \sqrt{(\Delta a/\bar{a})^2 + \Delta e^2 + (2\sin(\Delta i/2))^2} \tag{4}$$

with reject threshold $D > 2.0$. Currently deployed in *shadow mode*: the gate logs hypothetical rejections but does not discard pairs, enabling safe validation before activation.

Fail-Open Path. Objects with invalid elements, $e > 0.2$, or radial span exceeding bin coverage bypass all cheap filters and are checked against every debris object. Detailed fail-open counters are exposed in `/api/status?details=1`.

4.3 Narrow-Phase Collision Screening

Candidate pairs from the broad phase enter a multi-gate narrow pipeline:

1. **Plane/Phase Angular Gate.** Angular momentum vectors computed from (i, Ω). Pairs with orbital plane angle $> 75°$ or mean anomaly phase separation $> 150°$ are rejected. Objects with $e > 0.2$ bypass this gate (fail-open).
2. **MOID Proxy.** A sampled orbit–orbit minimum distance using a 72×72 true-anomaly grid (5,184 evaluations per pair). Reject threshold: 2.0 km. Deployed in shadow mode for safe rollout validation.
3. **Linear TCA Approximation.** Four simultaneous linear-segment minimum-distance estimates (start velocities, end velocities backward, averaged, and secant). The minimum d^2 across all four approximations determines whether the pair passes.
4. **RK4 Micro-Refinement.** Pairs within 0.10 km above the screening threshold receive a fine-grained RK4 propagation ($\Delta t_{\max} = 5\,\text{s}$).
5. **Full-Window RK4 Refinement.** Pairs within 0.20 km receive a budget-limited, OpenMP-parallelized full-window refinement (16 samples, 1 s substep). Thread-safe atomic budget decrement prevents compute runaway: base budget 64, dynamically scaled by pair count and step size, clamped to $[8, 192]$.

Uncertainty promotion adds 0.10 km to the screening band for pairs with relative speed exceeding 8 km/s. The

effective screening distance is:

$$d_{\text{screen}} = \underbrace{0.100}_{\text{collision}} + \underbrace{0.020}_{\text{TCA guard}} = 0.120 \text{ km} \tag{5}$$

4.4 Maneuver Planning

4.4.1 Autonomous COLA

When a predictive CDM record reaches CRITICAL severity ($d < 100\,\text{m}$), the engine triggers an autonomous evasion burn search over a discrete grid:

- 4 burn-time fractions of the remaining lead time
- 4 ΔV scales: $\{0.5, 1, 2, 4\} \times 0.001\,\text{km/s}$
- 10 RTN directions: $\pm T$, $\pm R$, 4 diagonal T/R, $\pm N$

The RTN frame is constructed as:

$$\hat{r} = \frac{\vec{r}}{|\vec{r}|}, \quad \hat{n} = \frac{\vec{r} \times \vec{v}}{|\vec{r} \times \vec{v}|}, \quad \hat{t} = \hat{n} \times \hat{r} \tag{6}$$

Candidates are ranked by a multi-criteria comparator: safety class (miss distance relative to target 0.15 km) $\rightarrow$ $|\Delta V|$ $\rightarrow$ normal component magnitude $\rightarrow$ slot error $\rightarrow$ predicted miss distance. The top 3 critical records per satellite are evaluated.

4.4.2 Fuel Accounting

Propellant expenditure follows the Tsiolkovsky relation:

$$\Delta m = m_{\text{current}} \left(1 - e^{-|\Delta\vec{v}|/v_e}\right), \quad v_e = I_{\text{sp}} \cdot g_0 \tag{7}$$

with $I_{\text{sp}} = 300\,\text{s}$, $g_0 = 9.80665\,\text{m/s}^2$, $m_{\text{dry}} = 500\,\text{kg}$, $m_{\text{fuel,init}} = 50\,\text{kg}$. A 5% fuel guard (2.5 kg) triggers automatic graveyard orbit transfer.

4.4.3 Constraint Enforcement

- **Cooldown**: 600 s mandatory rest between burns on the same satellite.
- **Thrust cap**: $|\Delta V| \leq 15\,\text{m/s}$ per burn.
- **LOS validation**: Upload windows scanned in 20 s steps across a 1,800 s horizon against 6 ground stations. 10 s signal latency enforced.
- **Blackout handling**: Burns must be computed and queued before the satellite leaves the last ground-station footprint.

4.5 Recovery Planning (CW/ZEM)

Every evasion maneuver triggers a paired recovery burn computed via the Clohessy–Wiltshire (CW) state-transition formulation. The default solver (`CW_ZEM_EQUIVALENT`) inverts the in-plane 2×2 Φ_{rv} submatrix:

$$\Delta\vec{v}_{\text{rec}} = \alpha\, \Phi_{rv}^{-1}\, \delta\vec{r} + \beta\, \delta\vec{v} \tag{8}$$

with $\alpha = 0.70$ (position correction weight), $\beta = 0.30$ (velocity damping weight), CW horizon $= 0.25 \times T_{\text{orbit}}$ clamped to $[300, 5400]$ s. The correction is capped at 15% of remaining ΔV budget and 40% of heuristic norm to prevent over-correction. A heuristic fallback (proportional RTN gains) activates when the CW matrix is near-singular ($nt \approx k \cdot 2\pi$).

The station-keeping box is defined as a 10 km spherical radius around the nominal orbital slot. Slot references are bootstrapped from first telemetry and J2-secularly propagated. The regression gate `recovery_slot_gate` enforces $\Delta_{\text{slot}} \leq 0.08$ deterministically.

5. Frontend Mission Console

The frontend is a React 18 / TypeScript single-page application built with Vite and rendered via Canvas 2D and Three.js WebGL. It polls the engine's REST API at 1–2 Hz and presents six operational views:

1. **Command Center** (`#/command`): Fleet-wide posture summary with satellite counts by status, active CDM warnings, recent burn history, and engine health.
2. **Ground Track** (`#/track`): Mercator-projection canvas rendering all satellite markers with 90-minute historical trails, 90-minute forecast paths, live solar terminator overlay, ground station markers, and a persistent map legend with epoch timestamp.
3. **Threat Assessment** (`#/threat`): Polar "bullseye" plot of debris in the selected satellite's proximity. Radial distance encodes time-to-closest-approach; angular position encodes approach vector. Color-coded by CDM severity (green/yellow/red).
4. **Burn Operations** (`#/burn-ops`): Maneuver timeline showing evasion and recovery burns with blackout zone overlays and cooldown friction markers. Fuel gauges per satellite.
5. **Fleet Status** (`#/fleet-status`): Aggregated constellation health with fuel heatmaps, ΔV-cost versus collisions-avoided plots, and station-keeping metrics.
6. **Scorecard** (`#/scorecard`): Judge-facing evidence dashboard rolling up all six PS §7 criteria with weighted readiness percentage, live evidence matrix, and submission gap tracking.

Rendering uses requestAnimationFrame with Canvas 2D for the ground track and Three.js for the 3D polar view. The snapshot endpoint delivers geodetic coordinates and a compressed debris tuple array (`[id, lat, lon, alt]`) to minimize payload size and maintain 60 FPS at 10,000+ debris markers.

6. Validation and Evidence

6.1 Test Infrastructure

CASCADE maintains 8 CTest gate tests, each a standalone C++ binary in `tools/` driven by shell harnesses in `scripts/`. All gates are enforced in GitHub Actions CI, and the Docker build itself fails if any test fails.

6.2 Real-Data Validation

Beyond synthetic test suites, CASCADE was validated against a real Space-Track OMM cata-

Gate	Invariant
`api_contract_gate`	All 5 endpoints conform to PS schema
`broad_phase_validation`	Shell filter $\supseteq$ brute-force baseline
`narrow_phase_false_negative`	Zero missed collisions vs. RK4 reference
`narrow_phase_calibration`	Screening thresholds within tolerance
`phase2_regression`	Propagator drift $<$ 0.061 km at 86,400 s
`maneuver_ops_invariants`	Cooldown, fuel, thrust-cap enforcement
`recovery_planner_invariants`	CW/ZEM convergence, slot error ≤ 0.08
`recovery_slot_gate`	Deterministic slot recovery

Table 2: CTest safety gates and their enforced invariants.

log (`data.txt`, 34 MB). The scenario generator (`tools/real_data_scenario_gen`) parsed 30,000+ objects via simdjson, filtered to LEO (periapsis $<$ 2,000 km altitude), classified payloads as satellites and the remainder as debris, and ran simulation ticks at full scale. Result: **27,191** LEO objects loaded in <1 s with zero ECI conversion failures, and 10 collisions detected against the real catalog (2.1 s/tick at full 27K scale).

6.3 Safety Score Evidence

- **False negatives**: 0 across 6 test families (broad-phase sanity, narrow-phase canary, false-negative gate, calibration probe, regression, maneuver-ops).
- **Fail-open counters**: Tracked per tick. Every object with uncertain elements is escalated, never discarded.
- **Screening threshold**: Effective 0.120 km with uncertainty promotion to 0.220 km for hypervelocity pairs.

6.4 Fuel Efficiency Evidence

- Tsiolkovsky fuel accounting validated in `maneuver_ops_invariants_gate`.
- CW/ZEM recovery solver minimizes $|\Delta V|$ relative to the heuristic proportional controller.
- Avoidance fuel vs. total fuel tracked separately in burn summaries.

6.5 Algorithmic Speed Evidence

Scenario	Mean Tick	Scale
50 sats / 10,000 debris	13.3 ms	Baseline
100 sats / 20,000 debris	72.6 ms	2× stress

Table 3: Tick latency benchmarks (`phase3_tick_benchmark`).

The broad phase reduces candidate count from $O(N \cdot M)$ to a small fraction via SMA/inclination binning. OpenMP parallelization of the narrow-phase sweep yields a 31.5% speedup at 2× scale. At full real-catalog scale (27,191 LEO objects), tick latency is 2.1 s — dominated by the sheer pair count but still well within the interactive demonstration budget.

Performance optimizations. Hot-path profiling identified `std::pow(rmag, 5.0)` in the J2 acceleration as a 10–50× bottleneck versus explicit multiplication (`r2*r2*rmag`). Precomputing shared J2 secular terms ($\cos i$, $\sin i$, $a^{3.5}$) across the three rate functions eliminated redundant transcendentals. The MOID sample count was increased from 24 to 72 after analysis showed that 15° resolution left 1,780 km arc gaps at LEO altitudes, rendering the gate effectively a no-op.

6.6 Deployment Verification

The Docker API smoke gate (`scripts/docker_api_smoke.sh`) performs an automated release-verification cycle:

1. Build Docker image from `Dockerfile`
2. Boot container on auto-selected port
3. Probe `/api/status`, POST telemetry, POST simulate/step, GET snapshot
4. Validate each response with Python assertions
5. Tear down container

This gate runs in CI and catches regressions before any push reaches main.

7. Deployment

7.1 Docker Build

CASCADE uses a 3-stage multi-stage Dockerfile:

Stage	Base	Purpose
builder	`ubuntu:22.04`	GCC-12, CMake, CTest gates
frontend	`node:24-slim`	`npm ci` + Vite production build
runtime	`ubuntu:22.04`	Minimal: `libgomp1` only

Table 4: Docker build stages.

BuildKit cache mounts accelerate rebuilds (`apt`, `_deps`, `sccache`). A `PROJECTBONK_PREFETCH_ONLY` step prefetches all FetchContent dependencies before source copy, maximizing layer-cache hits. The runtime image runs as a non-root user (`bonk`).

7.2 Quick Start

```
# Build and run
docker build -t cascade:local .
docker run --rm -p 8000:8000 cascade:local

# Smoke test
curl -s http://localhost:8000/api/status \
  | python3 -m json.tool

# Full verification
./scripts/docker_api_smoke.sh
./scripts/run_ready_demo.sh
```

7.3 CI Pipeline

GitHub Actions runs on every push to `main`:
1. CMake configure + build (`gcc-12`, Release)
2. CTest: all 8 safety gates
3. Frontend: `npm ci && npm run build`
4. Docker API smoke gate

8. Limitations and Future Work

- **Propagation fidelity**: The engine uses J2-only perturbation. Atmospheric drag is applied heuristically for perigee $< 350\,\text{km}$ orbits. Lunar/solar third-body and SRP effects are not modeled. For the hackathon's days-level prediction horizon, J2 dominates all other perturbations by 2–3 orders of magnitude, so this is an acceptable simplification.
- **MOID solver**: The current 72×72 sampled proxy achieves sufficient accuracy for screening but lacks the sub-meter precision of a Gronchi (2005) analytical quartic solver. We implemented the analytical path but encountered edge-case fragility on near-circular orbits (undefined ω); the sampled proxy is robust to all orbit types.
- **Maneuver optimization**: The grid-search COLA planner explores a discrete 160-candidate space (4 times $\times$ 4 ΔV $\times$ 10 directions). A continuous NLP or sequential convex programming formulation would yield lower-ΔV evasion sequences, but the discrete search completes in $< 1\,\text{ms}$ and gives deterministic, auditable results.
- **Dense-scene FPS**: Frontend rendering at 10,000+ debris has been tested in development but formal 60 FPS proof under worst-case load needs finalization.
- **Filter activation**: The D-criterion, plane/phase, and MOID gates are deployed in shadow mode. Activating them would reduce narrow-phase compute by an estimated 40–60% but requires additional calibration against adversarial edge cases to maintain the zero-false-negative guarantee.

Acknowledgments

We thank the organizers of the National Space Hackathon 2026 at IIT Delhi for designing a challenging and technically rigorous problem statement. Space-Track (`space-track.org`) provided the real OMM catalog used for full-scale validation.

A. Key Physical Constants

Symbol	Description	Value
μ	Earth grav. parameter	$398\,600.4418\,\text{km}^3/\text{s}^2$
R_E	Earth equatorial radius	$6\,378.137\,\text{km}$
J_2	Oblateness coefficient	1.08263×10^{-3}
d_{coll}	Collision threshold	$0.100\,\text{km}$
I_{sp}	Specific impulse	$300\,\text{s}$
g_0	Standard gravity	$9.80665\,\text{m/s}^2$
m_{dry}	Satellite dry mass	$500\,\text{kg}$
m_{fuel}	Initial propellant	$50\,\text{kg}$
$\lvert\Delta V\rvert_{\max}$	Thrust limit	$15\,\text{m/s}$
τ_{cd}	Cooldown period	$600\,\text{s}$
m_{EOL}	Fuel EOL guard	$2.5\,\text{kg}$
τ_{sig}	Signal latency	$10\,\text{s}$
r_{box}	Station-keeping box	$10\,\text{km}$

Table 5: System constants (from `src/types.hpp`).

B. CDM Severity Tiers

Tier	Condition	Auto-COLA
CRITICAL	$d < 0.100\,\text{km}$	Yes
WARNING	$d < 1.0\,\text{km}$	No (monitor)
WATCH	$d < 5.0\,\text{km}$	No (log)

Table 6: Conjunction severity classification.

C. Verification Commands

```
# Build (local)
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release
cmake --build build --target ProjectBONK

# Run all safety gates
ctest --test-dir build --output-on-failure

# Individual gates
./scripts/api_contract_gate.sh ./build
./scripts/broad_phase_sanity_gate.sh
./scripts/narrow_phase_false_negative_gate.sh
./scripts/recovery_planner_invariants_gate.sh

# Docker release verification
./scripts/docker_api_smoke.sh

# Frontend build
cd frontend && npm ci && npm run build

# Ready demo
./scripts/run_ready_demo.sh
```

D. NSGA-II Tuner Parameters

The offline multi-objective tuner implements the NSGA-II algorithm (Deb et al., 2002) via a Julia/`jluna` bridge. The optimizer drives 69 parameters across 3 tiers:

- **Tier 1 — Broad phase** (9 params): SMA bin width, inclination bin width, shell margin, neighbor search range, D-criterion threshold, fail-open eccentricity cutoff.
- **Tier 2 — Narrow phase** (21 params): plane/phase angular thresholds, MOID grid resolution and reject threshold, TCA guard margin, micro-refinement band, full-refine budget tiers, uncertainty promotion velocity cutoff.
- **Tier 3 — Maneuver planner** (39 params): COLA ΔV scales, RTN direction weights, CW horizon bounds, position/velocity correction blend, recovery budget cap, CDM scanner substep and horizon.

Three objectives are minimized simultaneously: *safety risk* (number of false negatives; any > 0 renders the solution infeasible via feasibility-first constraint handling), *fuel consumption* (total ΔV across the con-

stellation), and *tick latency* (wall-clock time for one simulation step). Verified end-to-end: 20 population $\times$ 5 generations (120 evaluations), all candidates safe (safety_risk = 0), best tick 13.3 ms from 23 ms baseline defaults.

All 69 constants are overridable at runtime via `PROJECTBONK_*` environment variables, enabling parameter sweeps without recompilation.

E. Development Timeline

The project was developed over approximately 4 weeks across 60+ commits. Table 7 highlights the major milestones.

Phase	Milestone
Week 1	Telemetry ingestion, SoA state store, J2 propagator, Kepler solver, basic REST endpoints
Week 2	Conservative broad-phase (shell + D-criterion), narrow-phase pipeline (TCA + MOID + RK4 refine), maneuver math (CW/ZEM), fuel and cooldown logic
Week 3	Shadow-gate observability, NSGA-II tuner, real-data scenario generator (27K objects), OpenMP parallelization, 69-param env-override system
Week 4	Frontend dashboard (6 pages), 3D→2D pivot, Docker multi-stage build, CI gates, PS alignment, technical report

Table 7: Development timeline and major milestones.